



Univerzitet u Novom Sadu
Tehnički fakultet
»MihajloPupin«
Zrenjanin



Školska 2025/26 godina
Nastavni predmet: Razvoj višeslojnog softvera

SEMINARSKI RAD

Tema: Prijava kandidata i obrada konkursa za posao

Predmetni nastavnik:
Prof. dr Ljubica Kazi
Predmetni asistent:
MSc Vuk Amižić

Student:
David Kiš 45/22

Zrenjanin, 2026. godina

Sadržaj:

1. Opis namene aplikacije u kontekstu poslovnog procesa i poslovnog pravila.....	3
2. Dokument i analiza dokumenta	5
3. Kratak opis primenjenih alata u izradi seminarskog rada.....	7
4. Modeli dokumentovanja realizovanog rešenja	8
4.1. Dijagram slučajeva korišćenja.....	8
4.2. Relacioni model podataka.....	9
4.3. Dijagram komponenti	10
4.4. Dijagram klasa.....	11
4.5. Dijagram sekvenci	12
5. Ključni delovi koda sa objašnjenjem	14
5.1. Realizacija korisničkih softverskih funkcija	14
5.1.1. Prijava korisnika	14
5.1.2. CRUD operacije nad glavnom tabelom	18
5.1.3. Štampa i parametarska štampa	24
5.2. Realizacija osnovnih validacija	26
5.3. Prezentacioni sloj – MVC i ViewModel	27
5.4. Sloj poslovne logike.....	29
5.5. Sloj servisa – REST servis.....	31
5.6. Sloj za rad sa podacima.....	33
5.6.1.1. Repository pattern	33
5.6.1.2. Rad sa stored procedurama.....	35
5.6.1.3. Rad sa DBUtils i primena upita.....	38
5.6.1.4. Rad sa Entity Framework klasama	39
5.7. Master-detail odnos podataka.....	40
5.8. JavaScript validacije i regularni izrazi.....	40

1. Opis namene aplikacije u kontekstu poslovnog procesa i poslovnog pravila

Namena aplikacije

Aplikacija „**Sistem za prijavu kandidata i obradu konkursa za posao**“ namenjena je elektronskom podnošenju i obradi prijava kandidata na konkurse za posao.

Sistem omogućava korisnicima da se prijave na sistem, unesu lične, kontaktne i obrazovne podatke, izaberu ili unesu stečeno zanimanje i radno mesto za koje se prijavljuju, kao i da evidentiraju dokumentaciju dostavljenu uz prijavu.

Administrator sistema ima mogućnost pregleda svih prijava, pretrage i filtriranja prijava, pregleda detalja, izmene i brisanja prijava, kao i prihvatanja ili odbijanja kandidata.

Sistem je realizovan kao višeslojna ASP.NET MVC aplikacija uz korišćenje relacionih baza podataka, stored procedura, poslovne logike, REST servisa, XML datoteka i JavaScript funkcionalnosti.

Poslovni proces

Poslovni proces funkcioniše kroz sledeće korake:

1. Korisnik se prijavljuje na sistem pomoću korisničkog imena i lozinke.
2. Korisnik otvara obrazac za podnošenje nove prijave.
3. Korisnik unosi lične i kontaktne podatke.
4. Korisnik unosi podatke o poslednjoj završenoj školi, mestu škole, stečenom zanimanju i stepenu obrazovanja.
5. Korisnik unosi naziv konkursa i radno mesto za koje se prijavljuje.
6. Sistem učitava ponuđena zanimanja i radna mesta iz XML datoteke.
7. Korisnik bira ili ručno unosi zanimanje i radno mesto.
8. Korisnik unosi godine radnog iskustva, rok konkursa i motivaciono pismo.
9. Korisnik označava dokumentaciju koju dostavlja uz prijavu.
10. Sistem proverava da li je prijava podneta u okviru roka konkursa.
11. Sistem proverava da li stečeno zanimanje odgovara radnom mestu prema evidenciji usklađenosti iz XML datoteke.
12. Na osnovu poslovnog pravila sistem automatski određuje početni status prijave.
13. Prijava i prateća dokumentacija čuvaju se u bazi podataka.
14. Administrator pregleda sve prijave kandidata.
15. Administrator može da pretražuje i filtrira prijave prema statusu i drugim podacima.
16. Administrator može da pregleda detalje, izmeni ili obriše prijavu.
17. Administrator donosi konačnu odluku i kandidata označava kao izabranog ili odbijenog.
18. Sistem omogućava pregled i štampu pojedinačne prijave ili spiska prijava.

Poslovno pravilo

Poslovno pravilo u sistemu glasi:

AKO je prijava podneta u okviru roka konkursa I stečeno zanimanje kandidata je u evidenciji označeno kao odgovarajuće ili srodno radnom

mestu za koje je konkurs raspisan

ONDA se prijava automatski postavlja u status „U razmatranju“.

AKO je rok konkursa istekao, ONDA se prijava postavlja u status „Odbijena“.

2. Dokument i analiza dokumenta

Prikaz dokumenta

Konkursi za posao

Početna Sve prijave Odjava

DETALJI PRIJAVE KANDIDATA

David Kis

Konkurs: Data engineering

Odbijena

Podaci o kandidatu

Ime i prezime

David Kis

E-mail

kontrash297@gmail.com

Telefon

0605857087

Posljednja završena škola

Tehnička Škola Becej

Mesto škole

Becej

Stekeno zanimanje

Elektrotehničar informacionih tehnologija

Stepen obrazovanja

Srednja škola

Godine iskustva

0

Podaci o prijavi

Radno mesto

Junior programer

Datum podnošenja

16.06.2026

Rok konkursa

01.01.0001

Status prijave

Odbijena

Ispunjava osnovne uslove

Ne

Motivaciono pismo

Slika 1. – Stranica Detalji

Godine iskustva

0

Motivaciono pismo

Nije uneto motivaciono pismo.

Dokumentacija

Biografija (CV)

Dostavljen

Motivaciono pismo

Dostavljen

Diploma ili potvrda o obrazovanju

Dostavljen

Preporuka prethodnog poslodavca

Dostavljen

Nazad

Izmeni prijavu

Štampaj prijavu

© 2026 - Sistem za prijavu kandidata i obradu konkursa

Slika 2. – Stranica Detalji

Naziv podatka	Tip podatka	Domen
ZahtevID	int	Pozitivan ceo broj, primarni ključ, automatski se generiše
KorisnikID	int	Pozitivan ceo broj, strani ključ ka tabeli <i>Korisnici</i>
ImePrezime	nvarchar(100)	Tekst do 100 karaktera, obavezan podatak
Email	nvarchar(100)	Ispravna e-mail adresa do 100 karaktera, obavezan podatak
KontaktTelefon	nvarchar(20)	Broj telefona do 20 karaktera, obavezan podatak
NazivKonkursa	nvarchar(150)	Tekst do 150 karaktera, obavezan podatak
StepenObrazovanja	nvarchar(80)	Srednja škola, Viša škola, Visoko obrazovanje ili Master
GodineIskustva	int	Ceo broj od 0 do 50
MotivacionoPismo	nvarchar(max)	Proizvoljan duži tekst, nije obavezan podatak
DatumPodnosenja	date	Datum kada je prijava podneta
RokKonkursa	date	Krajnji datum za podnošenje prijave
StatusZahteva	nvarchar(30)	Primljena, U razmatranju, Odbijena ili Izabran
IspunjavaOsnovneUslove	bit	1 – kandidat ispunjava uslove, 0 – kandidat ne ispunjava uslove
PoslednjaSkola	nvarchar(150)	Naziv poslednje završene škole do 150 karaktera
MestoSkole	nvarchar(100)	Naziv mesta u kojem je škola završena do 100 karaktera
Zanimanje	nvarchar(150)	Stečeno zanimanje kandidata ili vrednost iz XML evidencije
RadnoMesto	nvarchar(150)	Radno mesto za koje se kandidat prijavljuje ili vrednost iz XML evidencije

Objašnjenje domena

Domen predstavlja skup dozvoljenih vrednosti i ograničenja koja važe za određeni podatak.

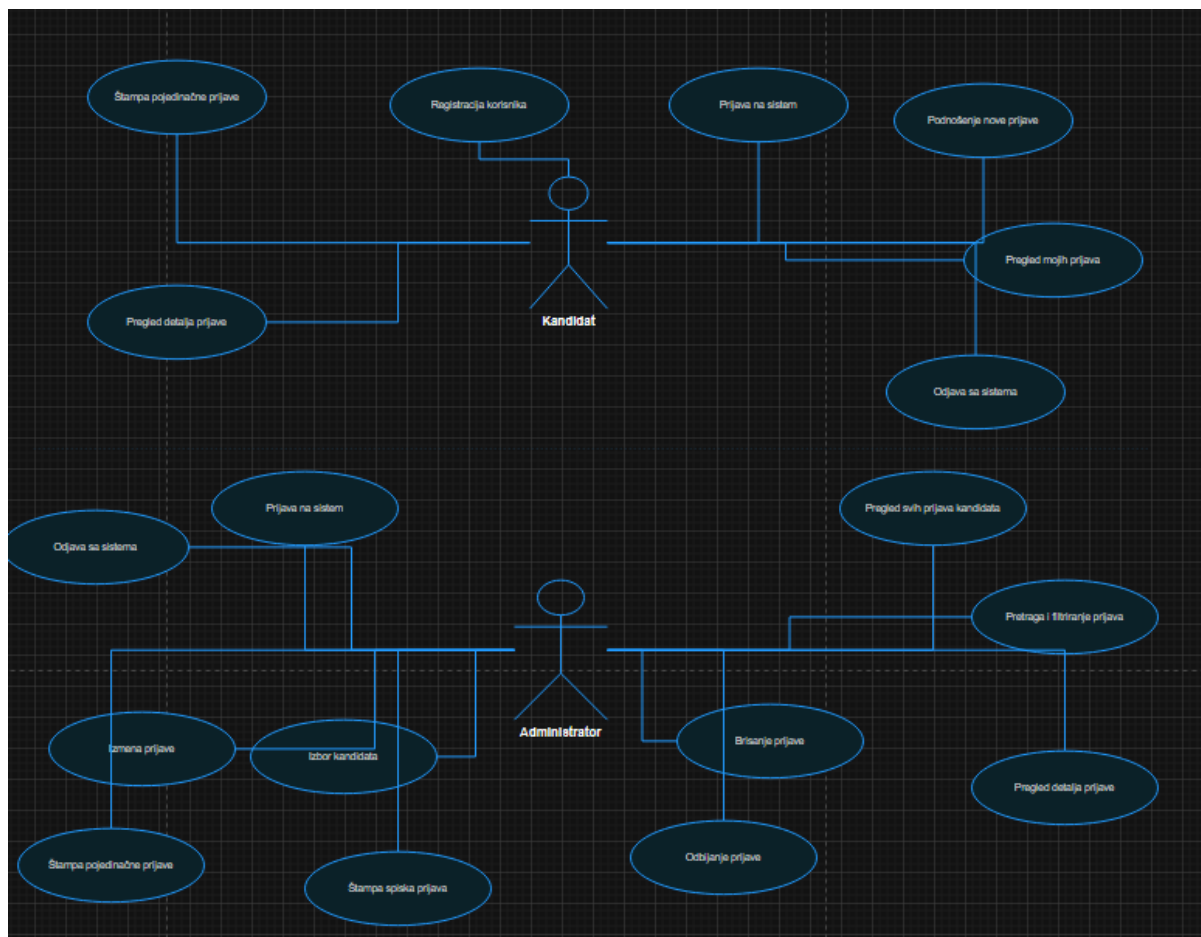
3. Kratak opis primenjenih alata u izradi seminarskog rada

Za realizaciju seminarskog rada korišćeni su sledeći alati i tehnologije:

- Visual Studio 2026
- ASP.NET MVC
- MS SQL Server
- Entity Framework
- REST servis
- Bootstrap
- C#
- HTML, CSS i JavaScript
- DBUtils biblioteka
- SQL Stored Procedure

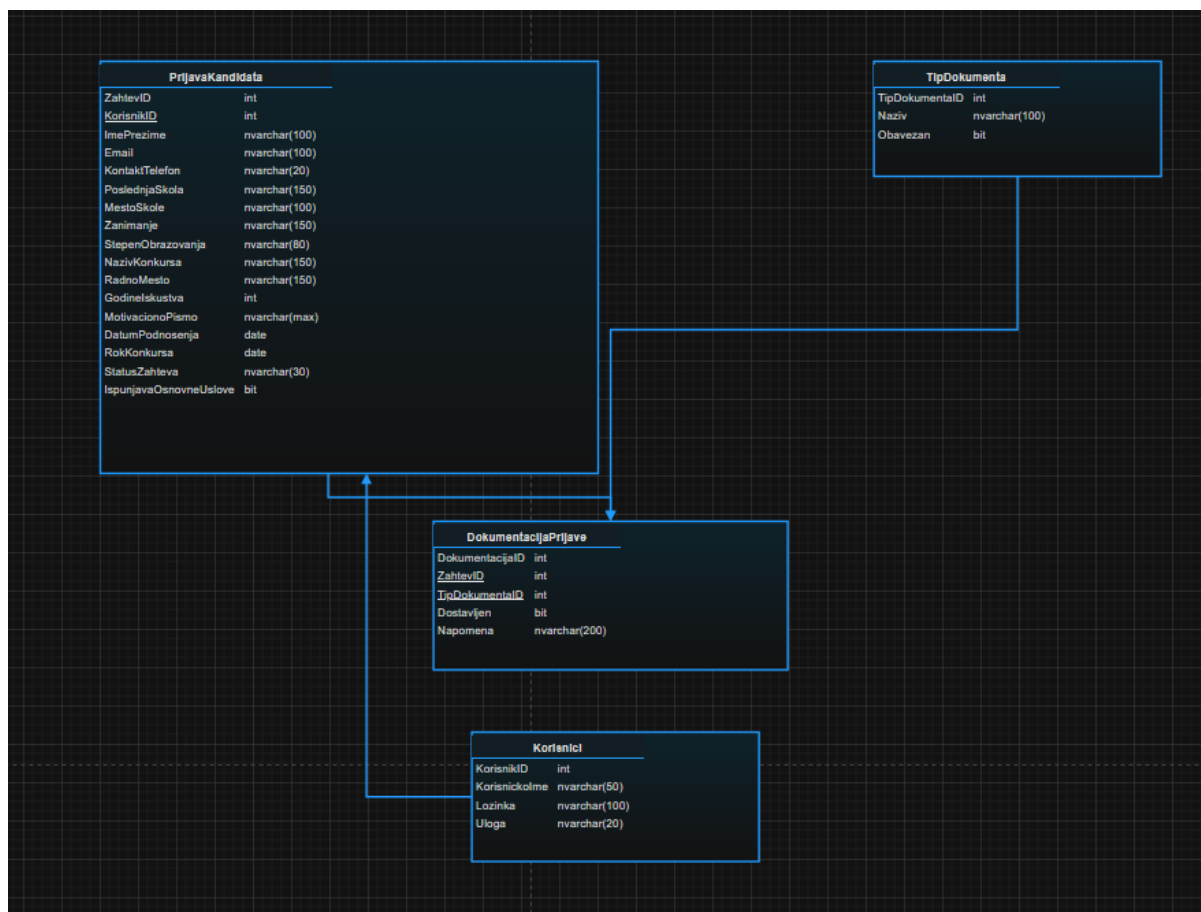
4. Modeli dokumentovanja realizovanog rešenja

4.1. Dijagram slučajeja korišćenja



Slika 3. – Dijagram slučajeja korišćenja Prijava kandidata i obrada konkursa za posao

4.2. Relacioni model podataka



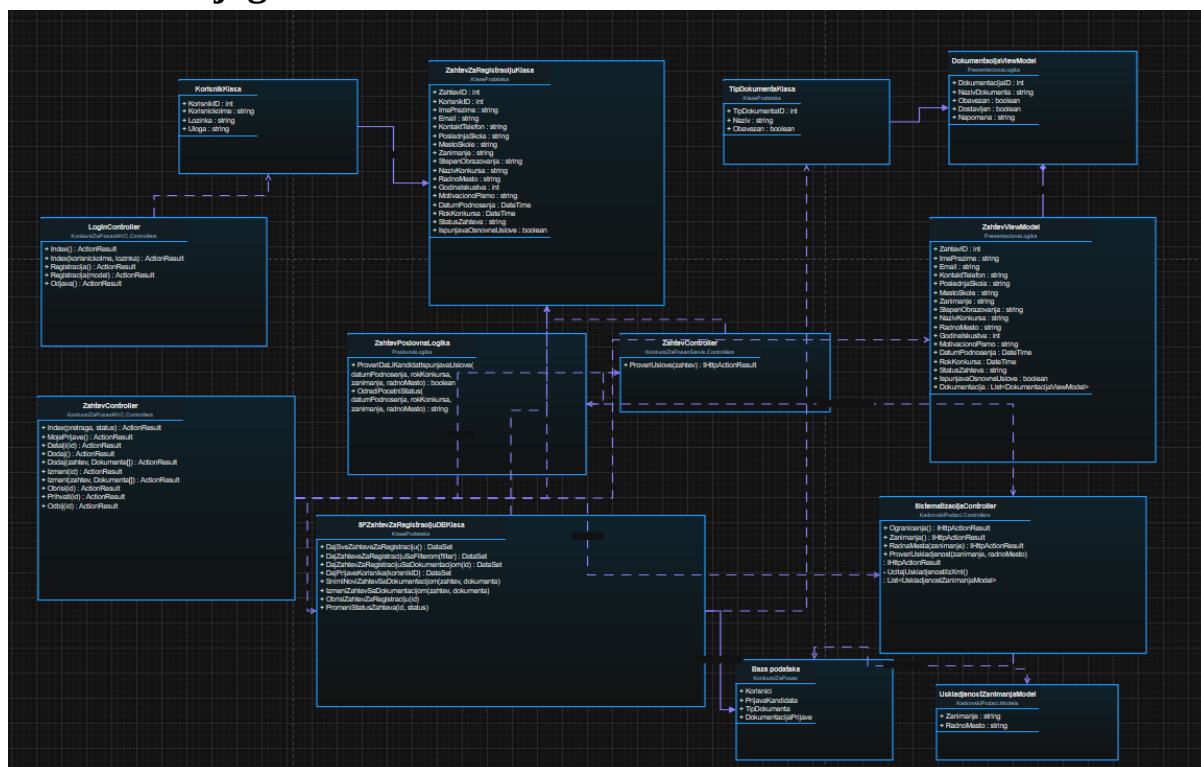
Slika 4. – Relacioni model baze podataka

4.3. Dijagram komponenti



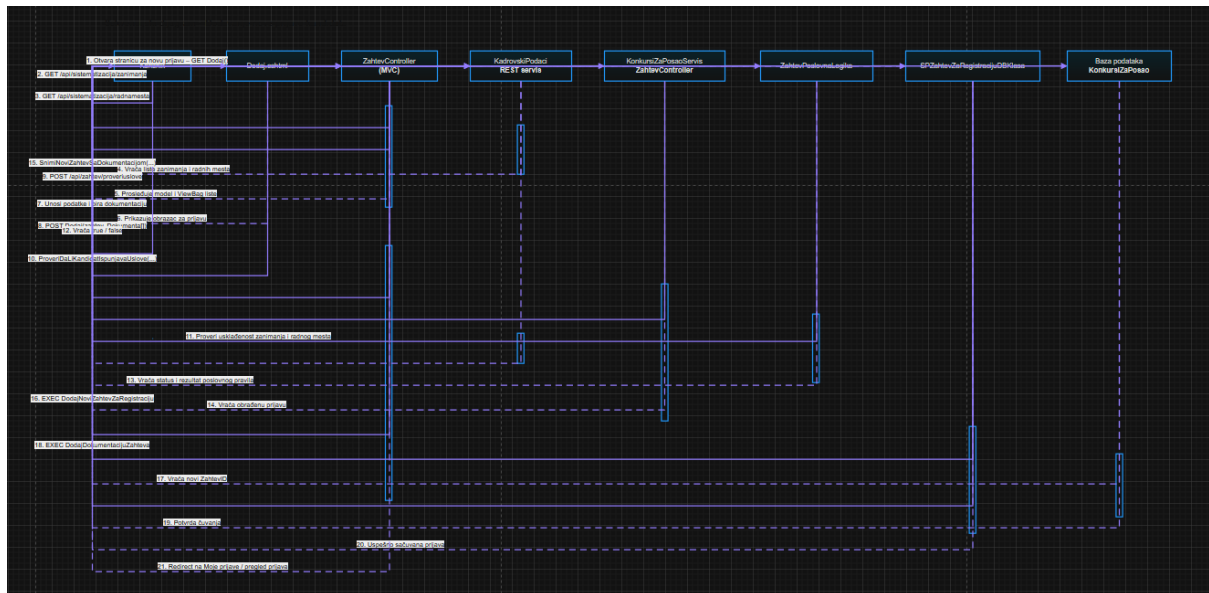
Slika 5. – Dijagram komponenti sistema

4.4. Dijagram klasa

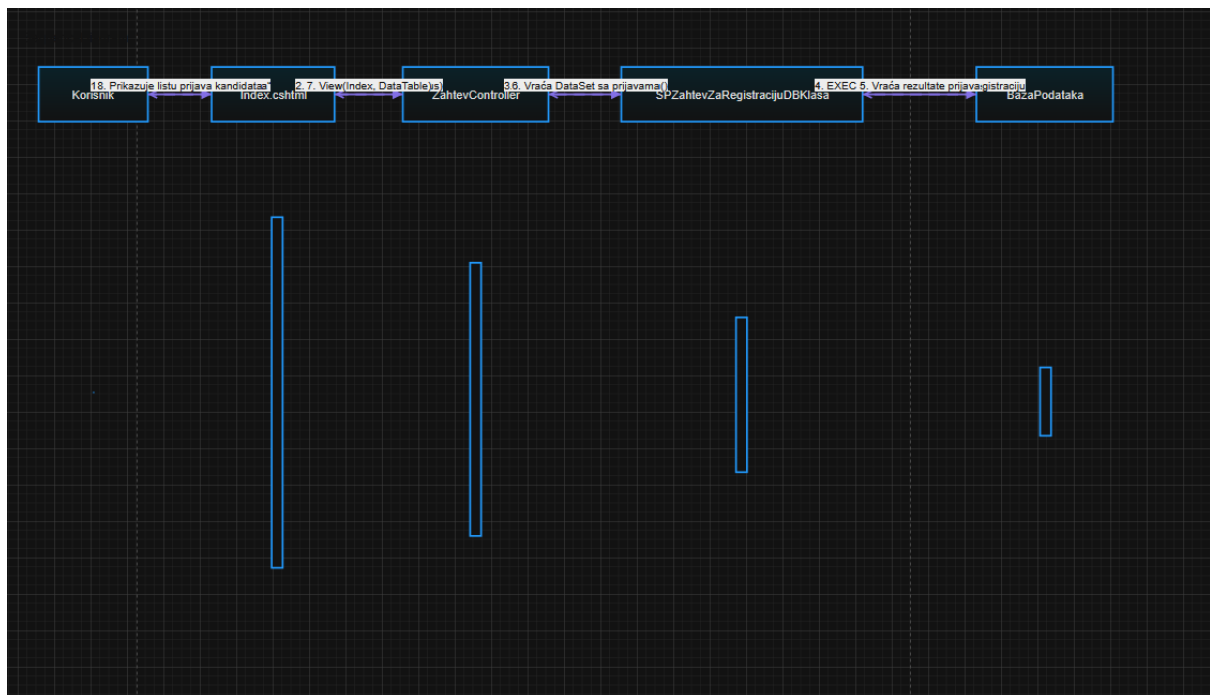


Slika 6. – Dijagram klasa realizovanog sistema

4.5. Dijagram sekvenci



Slika 7. - Dijagram sekvenci za unos zahteva



Slika 8. - Dijagram sekvenci za tabelarni prikaz zahteva

5. Ključni delovi koda sa objašnjenjem

5.1. Realizacija korisničkih softverskih funkcija

5.1.1. Prijava korisnika

Prijava korisnika realizovana je kroz Login kontroler i proveru korisničkog imena i lozinke iz baze podataka. Nakon uspešne prijave kreira se Session objekat.

```
using System;
using System.Web.Mvc;
using KlasePodataka;

namespace KonkursiZaPosaoMVC.Controllers
{
    public class LoginController : Controller
    {
        [HttpGet]
        public ActionResult Index()
        {
            return View();
        }

        [HttpPost]
        public ActionResult Index(
            string korisnickoIme,
            string lozinka)
        {
            EFKorisnikDBKlasa efKorisnikDb =
                new EFKorisnikDBKlasa();

            KorisnikKlasa korisnik =
                efKorisnikDb
                    .DajKorisnikaPoKorisnickomImenuILOzinci(
                        korisnickoIme,
                        lozinka
                    );

            if (korisnik == null)
            {
                ViewBag.Greska =
                    "Pogrešno korisničko ime ili lozinka.";
            }
        }
    }
}
```

```

        return View();
    }

    Session["KorisnikID"] =
        korisnik.KorisnikID;

    Session["Korisnik"] =
        korisnik.KorisnickoIme;

    Session["Uloga"] =
        korisnik.Uloga;

    return RedirectToAction(
        "Index",
        "Home"
    );
}

[HttpGet]
public ActionResult Registracija()
{
    return View();
}

[HttpPost]
public ActionResult Registracija(
    string korisnickoIme,
    string lozinka,
    string potvrdaLozinke)
{
    korisnickoIme =
        korisnickoIme == null
            ? string.Empty
            : korisnickoIme.Trim();

    if (string.IsNullOrEmpty(korisnickoIme)
        ||
        string.IsNullOrEmpty(lozinka)
        ||
        string.IsNullOrEmpty(potvrdaLozinke))
    {
        ViewBag.Greska =
            "Sva polja su obavezna.";

        return View();
    }
}

```

```

    }

    if (korisnickoIme.Length < 3)
    {
        ViewBag.Greska =
            "Korisničko ime mora imati najmanje 3 karaktera.";

        return View();
    }

    if (lozinka.Length < 4)
    {
        ViewBag.Greska =
            "Lozinka mora imati najmanje 4 karaktera.";

        return View();
    }

    if (lozinka != potvrdaLozinke)
    {
        ViewBag.Greska =
            "Lozinka i potvrda lozinke se ne podudaraju.";

        return View();
    }

    EFKorisnikDBKlasa efKorisnikDb =
        new EFKorisnikDBKlasa();

    KorisnikKlasa postojeciKorisnik =
        efKorisnikDb
            .DajKorisnikaPoKorisnickomImenu(
                korisnickoIme
            );

    if (postojeciKorisnik != null)
    {
        ViewBag.Greska =
            "Korisničko ime je već zauzeto.";

        return View();
    }

    KorisnikKlasa noviKorisnik =
        new KorisnikKlasa
        {
            KorisnickoIme =
                korisnickoIme,

```



```

        Lozinka =
            lozinka,

        Uloga =
            "Korisnik"
    };

    bool uspesno =
        efKorisnikDb.DodajKorisnika(
            noviKorisnik
        );

    if (!uspesno)
    {
        ViewBag.Greska =
            "Registracija nije uspela.";

        return View();
    }

    TempData["Poruka"] =
        "Registracija je uspešna. Sada se možete prijaviti.";

    return RedirectToAction(
        "Index"
    );
}

public ActionResult Odjava()
{
    Session.Clear();
    Session.Abandon();

    return RedirectToAction(
        "Index",
        "Home"
    );
}
}
}

```

Listing 1. – Kontroler za login korisnika

5.1.2. CRUD operacije nad glavnom tabelom

CRUD operacije realizovane su kroz ZahtevController.

Podržane operacije:

- Dodavanje zahteva
- Izmena zahteva
- Brisanje zahteva
- Pregled detalja
- Tabelarni prikaz

```
[HttpPost]
[ValidateAntiForgeryToken]
public ActionResult Dodaj(
    ZahtevZaRegistracijuKlasa zahtev,
    int[] Dokumenta)
{
    if (!KorisnikJePrijavljen())
    {
        return RedirectToAction(
            "Index",
            "Login"
        );
    }

    PopuniListeIzXml();

    zahtev.KorisnikID =
        Convert.ToInt32(
            Session["KorisnikID"]
        );

    ViewBag.IzabranaDokumenta =
        PretvoriDokumentaUListu(
            Dokumenta
        );

    if (!ModelState.IsValid)
    {
        return View(zahtev);
    }
}
```

```

}

try
{
    List<int> dokumenta =
        PretvoriDokumentaUListu(
            Dokumenta
        );

    PoslovnaLogika.ZahtevPoslovnaLogika poslovnaLogika =
        new PoslovnaLogika.ZahtevPoslovnaLogika();

    zahtev.DatumPodnosenja =
        DateTime.Today;

    zahtev.IspunjavaOsnovneUslove =
        poslovnaLogika.ProveriDaLiKandidatIspunjavaUslove(
            zahtev.DatumPodnosenja,
            zahtev.RokKonkursa,
            zahtev.Zanimanje,
            zahtev.RadnoMesto
        );

    zahtev.StatusZahteva =
        poslovnaLogika.OdrediPocetniStatus(
            zahtev.DatumPodnosenja,
            zahtev.RokKonkursa,
            zahtev.Zanimanje,
            zahtev.RadnoMesto
        );

    SPZahtevZaRegistracijuDBKlasa db =
        new SPZahtevZaRegistracijuDBKlasa(
            connectionString
        );

    db.SnimiNoviZahtevSaDokumentacijom(
        zahtev,
        dokumenta
    );

    TempData["Poruka"] =
        "Prijava je uspešno podneta.";

    if (KorisnikJeAdministrator())
    {
        return RedirectToAction(
            "Index",

```

```

        "Zahtev"
    );
}

return RedirectToAction(
    "MojePrijava",
    "Zahtev"
);
}
catch (Exception ex)
{
    string detaljnaGreska =
        ex.InnerException != null
            ? ex.InnerException.Message
            : ex.Message;

    ModelState.AddModelError(
        "",
        "Greška prilikom čuvanja prijave: " +
        detaljnaGreska
    );

    return View(zahtev);
}
}

```

Listing 2. – Dodaj metoda

```

[HttpPost]
[ValidateAntiForgeryToken]
public ActionResult Izmeni(
    ZahtevZaRegistracijuKlasa zahtev,
    int[] Dokumenta)
{
    if (!KorisnikJePrijavljen())
    {
        return RedirectToAction(
            "Index",
            "Login"
        );
    }

    if (!KorisnikJeAdministrator())
    {
        return new HttpStatusCodeResult(
            403,
            "Samo administrator može da menja prijave."
        );
    }

    PopuniListeIzXml();

    List<int> dokumenta =
        PretvoriDokumentaUListu(
            Dokumenta
        );

    ViewBag.IzabranaDokumenta =
        dokumenta;

    if (!ModelState.IsValid)
    {
        return View(zahtev);
    }

    try
    {
        PoslovnaLogika.ZahtevPoslovnaLogika poslovnaLogika =
            new PoslovnaLogika.ZahtevPoslovnaLogika();

        zahtev.IspunjavaOsnovneUslove =
            poslovnaLogika.ProveriDaLiKandidatIspunjavaUslove(
                zahtev.DatumPodnosenja,
                zahtev.RokKonkursa,
                zahtev.Zanimanje,

```

```

        zahtev.RadnoMesto
    );

    zahtev.StatusZahteva =
        poslovnaLogika.OdrediPocetniStatus(
            zahtev.DatumPodnosenja,
            zahtev.RokKonkursa,
            zahtev.Zanimanje,
            zahtev.RadnoMesto
        );

    SPZahtevZaRegistracijuDBKlasa db =
        new SPZahtevZaRegistracijuDBKlasa(
            connectionString
        );

    db.IzmeniZahtevSaDokumentacijom(
        zahtev,
        dokumenta
    );

    TempData["Poruka"] =
        "Prijava je uspešno izmenjena.";

    return RedirectToAction(
        "Index"
    );
}
catch (Exception ex)
{
    string detaljnaGreska =
        ex.InnerException != null
            ? ex.InnerException.Message
            : ex.Message;

    ModelState.AddModelError(
        "",
        "Greška prilikom izmene prijave: " +
        detaljnaGreska
    );

    return View(zahtev);
}
}

```

Listing 3. – “Izmeni” metod

```

public ActionResult Obrisi(int id)
{
    if (!KorisnikJePrijavljen())
    {
        return RedirectToAction(
            "Index",
            "Login"
        );
    }

    if (!KorisnikJeAdministrator())
    {
        return new HttpStatusCodeResult(
            403,
            "Samo administrator može da briše prijave."
        );
    }

    SPZahtevZaRegistracijuDBKlasa db =
        new SPZahtevZaRegistracijuDBKlasa(
            connectionString
        );

    db.ObrisiZahtevZaRegistraciju(
        id
    );

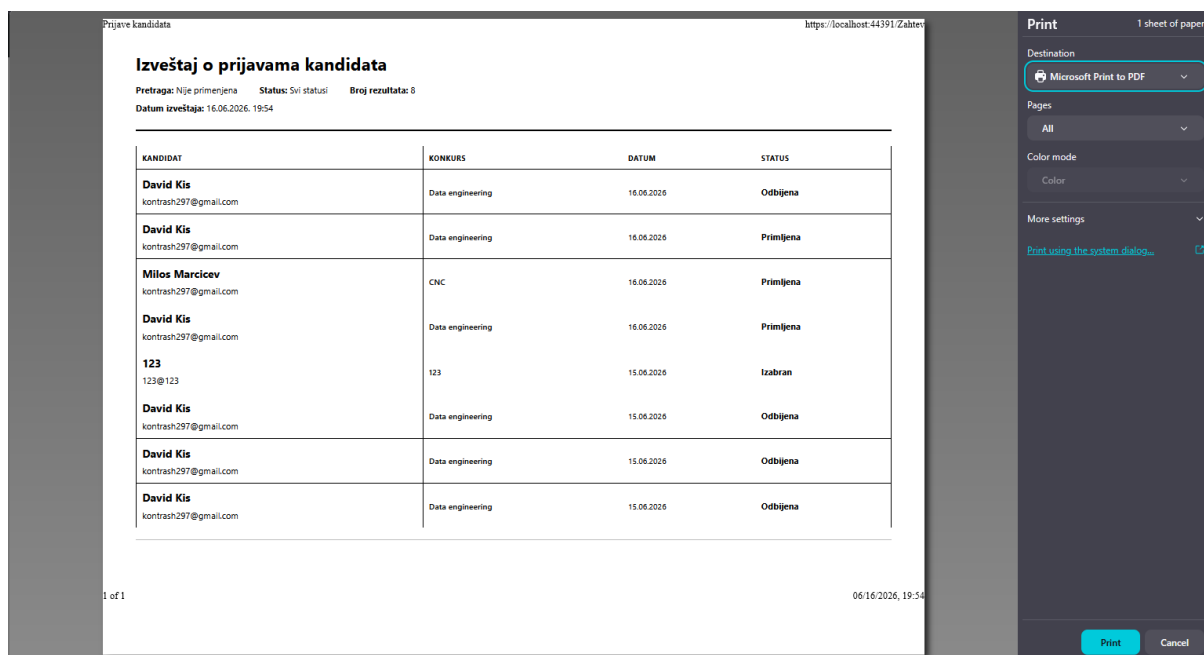
    return RedirectToAction(
        "Index"
    );
}

```

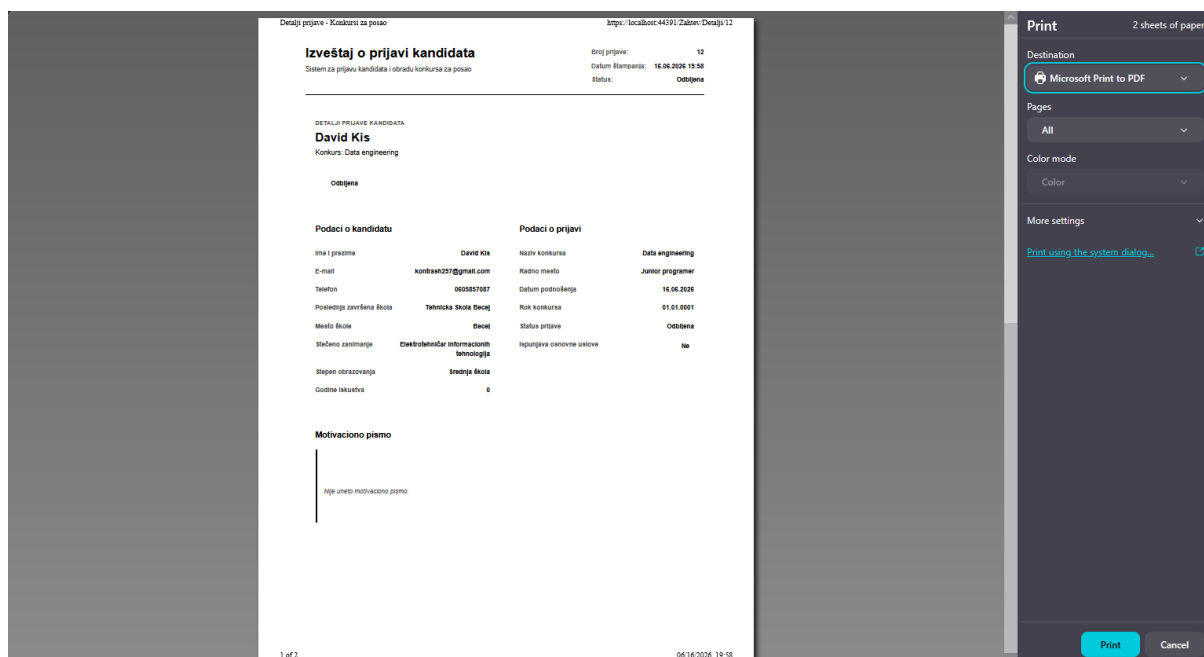
Listing 4. – Obriši metoda

5.1.3. Štampa i parametarska štampa

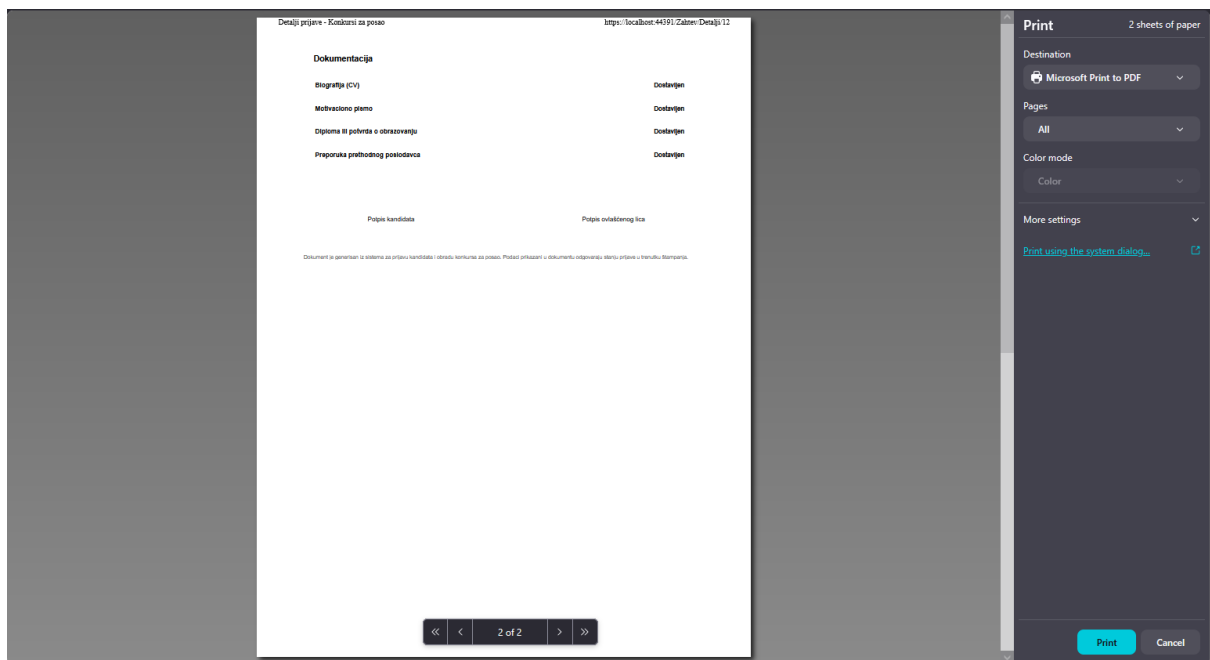
Štampa je realizovana korišćenjem printer-friendly stranice i JavaScript funkcije `window.print()`.



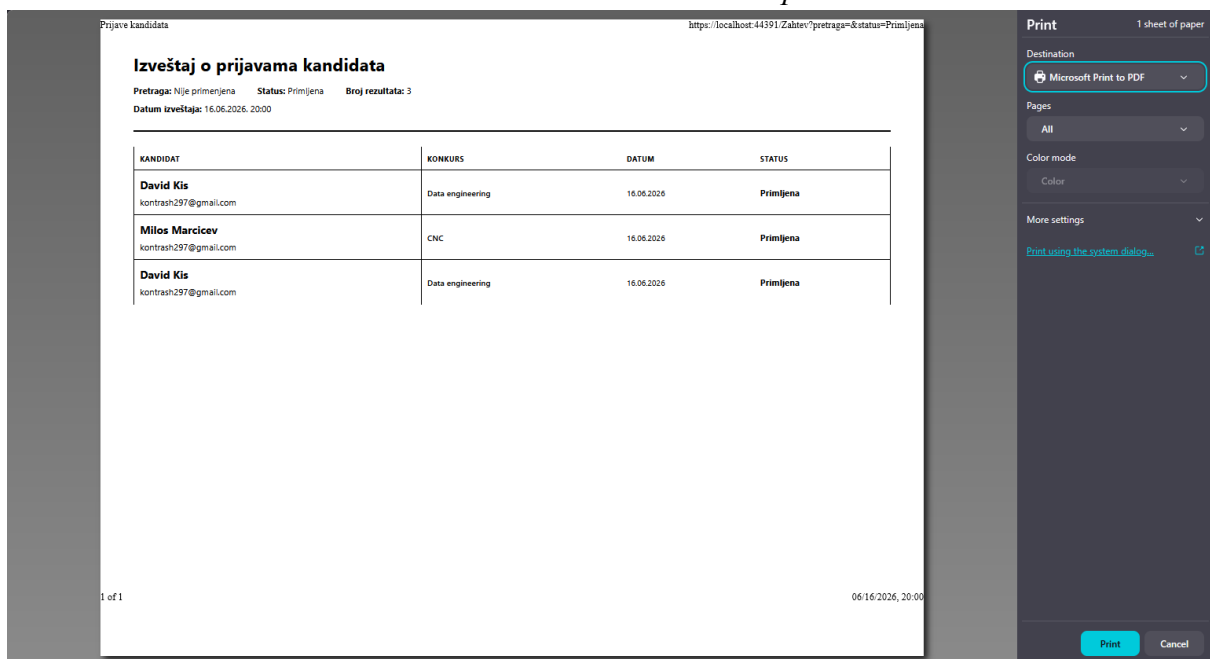
Slika 9. – Štampa liste o prijavama kandidata



Slika 10. – Parametarska štampa



Slika 11. – Parametarka štampa



Slika 12. – Štampa po filterima

5.2. Realizacija osnovnih validacija

Validacije se koriste za proveru podataka pre slanja forme serveru. Na primer, polje `GodineIskustva` mora sadržati ceo broj između 0 i 50. Ukoliko unos nije ispravan, slanje forme se zaustavlja, prikazuje se poruka o grešci i fokus se postavlja na neispravno polje. Ova provera predstavlja klijentsku validaciju i dopunjuje serversku validaciju definisanu u modelu.

Validacije su realizovane kroz:

- Required validacije
- Regularne izraze
- JavaScript proveru

```
document.getElementById("FormaPrijave")
    .addEventListener("submit", function (dogadjaj) {

        var poljeGodine =
            document.getElementById("GodineIskustva");

        var godine =
            Number(poljeGodine.value);

        if (
            poljeGodine.value.trim() === ""
            || !Number.isInteger(godine)
            || godine < 0
            || godine > 50
        ) {
            dogadjaj.preventDefault();

            alert(
                "Godine iskustva moraju biti ceo broj između 0 i 50."
            );

            poljeGodine.focus();
        }
    });
```

Listing 5. – JavaScript validacija

5.3. Prezentacioni sloj – MVC i ViewModel

Prezentacioni sloj realizovan je korišćenjem ASP.NET MVC arhitekture.

Korišćen je ZahtevViewModel za prikaz detalja zahteva i dokumentacije.

```
using System;
using System.Collections.Generic;

namespace Prezentacionalogika
{
    public class ZahtevViewModel
    {
        public int ZahtevID { get; set; }

        public string ImePrezime { get; set; }

        public string Email { get; set; }

        public string KontaktTelefon { get; set; }

        public string PoslednjaSkola { get; set; }

        public string MestoSkole { get; set; }

        public string Zanimanje { get; set; }

        public string NazivKonkursa { get; set; }

        public string RadnoMesto { get; set; }

        public string StepenObrazovanja { get; set; }

        public int GodineIskustva { get; set; }

        public string MotivacionoPismo { get; set; }

        public DateTime DatumPodnosenja { get; set; }

        public DateTime RokKonkursa { get; set; }

        public string StatusZahteva { get; set; }
    }
}
```

```

public bool IspunjavaOsnovneUslove { get; set; }

public List<DokumentacijaViewModel> Dokumentacija
{
    get;
    set;
}
}

```

Listing 6. – ViewModel klasa prezentacionog sloja

5.4. Sloj poslovne logike

Poslovna logika realizovana je kroz klasu `ZahtevPoslovnaLogika`.

Poslovnopravilo:

Ako je vozilo starije od definisane granice godina, sistem automatski označava potreban tehnički pregled na 6 meseci.

```
using System;

namespace PoslovnaLogika
{
    public class ZahtevPoslovnaLogika
    {
        public bool ProveriDaLiKandidatIspunjavaUslove(
            DateTime datumPodnosenja,
            DateTime rokKonkursa,
            string zanimanje,
            string radnoMesto)
        {
            bool prijavaJeUroku =
                datumPodnosenja.Date <=
                rokKonkursa.Date;

            if (!prijavaJeUroku)
            {
                return false;
            }

            WSKadrovskiPodaci
                .OgranicenjaZaposljavanja servis =
                new WSKadrovskiPodaci
                    .OgranicenjaZaposljavanja();

            servis.Url =
                "http://localhost:1718/" +
                "OgranicenjaZaposljavanja.asmx";

            return servis
                .DaLiZanimanjeOdgovaraRadnomMestu(
                    zanimanje,
                    radnoMesto
                );
        }

        public string OdrediPocetniStatus(
```

```

        DateTime datumPodnosenja,
        DateTime rokKonkursa,
        string zanimanje,
        string radnoMesto)
    {
        if (datumPodnosenja.Date >
            rokKonkursa.Date)
        {
            return "Odbijena";
        }

        bool zanimanjeOdgovara =
            ProveriDaLiKandidatIspunjavaUslove(
                datumPodnosenja,
                rokKonkursa,
                zanimanje,
                radnoMesto
            );

        if (zanimanjeOdgovara)
        {
            return "U razmatranju";
        }

        return "Primljena";
    }
}

```

Listing 7. – Poslovna pravila

5.5. Sloj servisa – REST servis

REST servis realizovan je u projektu KonkursiZaPosaoServis.

Servis se koristi za:

- proveru radnog mesta
- proveru zanimanja
- proveru ogranicenja
- čitanje parametara poslovnog pravila

```
[HttpGet]
[Route("proveriuskladjenost")]
public IHttpActionResult ProveriUskladjenost(
    string zanimanje,
    string radnoMesto)
{
    if (string.IsNullOrEmpty(zanimanje))
    {
        return BadRequest(
            "Zanimanje nije prosleđeno."
        );
    }

    if (string.IsNullOrEmpty(radnoMesto))
    {
        return BadRequest(
            "Radno mesto nije prosleđeno."
        );
    }

    try
    {
        bool uskladjeno =
            UcitajUskladjenostiIzXml()
                .Any(
                    stavka =>
                        string.Equals(
                            stavka.Zanimanje.Trim(),
                            zanimanje.Trim(),
                            StringComparison.OrdinalIgnoreCase
                        )
                        &&
                        string.Equals(

```

```

        stavka.RadnoMesto.Trim(),
        radnoMesto.Trim(),
        StringComparison.OrdinalIgnoreCase
    );

    return Ok(
        uskladjeno
    );
}
catch (Exception ex)
{
    return BadRequest(
        DajDetaljnuGresku(ex)
    );
}
}

```

Listing 8. – Realizacija REST servisa

5.6. Sloj za rad sa podacima

5.6.1.1. Repository pattern

Repository pattern realizovan je kroz klase:

- SPZahtevZaRegistracijuDBKlasa
- DBUtilsTipDokumentaKlasa

```
namespace KlasePodataka
{
    public class SPZahtevZaRegistracijuDBKlasa
    {
        private readonly string connectionString;

        public SPZahtevZaRegistracijuDBKlasa(
            string connectionString)
        {
            this.connectionString = connectionString;
        }

        private DataSet IzvrsiSelectProceduru(
            string nazivProcedure,
            params SqlParameter[] parametri)
        {
            DataSet dataSet = new DataSet();

            using (SqlConnection konekcija =
                new SqlConnection(connectionString))
            using (SqlCommand komanda =
                new SqlCommand(nazivProcedure, konekcija))
            {
                komanda.CommandType =
                    CommandType.StoredProcedure;

                if (parametri != null &&
                    parametri.Length > 0)
                {
                    komanda.Parameters.AddRange(parametri);
                }

                using (SqlDataAdapter adapter =
                    new SqlDataAdapter(komanda))
                {

```

```
        adapter.Fill(dataSet);  
    }  
}  
  
return dataSet;  
}
```

Listing 9. – Realizacija Repository patterna

5.6.1.2. Rad sa stored procedurama

Stored procedure koriste se za:

- unos
- izmenu
- brisanje
- filtriranje podataka

```
CREATE OR ALTER PROCEDURE dbo.DodajNoviZahtevZaRegistraciju
    @KorisnikID INT,
    @ImePrezime NVARCHAR(100),
    @Email NVARCHAR(100),
    @KontaktTelefon NVARCHAR(20),
    @PoslednjaSkola NVARCHAR(150),
    @MestoSkole NVARCHAR(100) = NULL,
    @Zanimanje NVARCHAR(150),
    @StepenObrazovanja NVARCHAR(80),
    @NazivKonkursa NVARCHAR(150),
    @RadnoMesto NVARCHAR(150),
    @GodineIskustva INT,
    @MotivacionoPismo NVARCHAR(MAX) = NULL,
    @DatumPodnosenja DATE,
    @RokKonkursa DATE,
    @StatusZahteva NVARCHAR(30),
    @IspunjavaOsnovneUslove BIT
AS
BEGIN
    SET NOCOUNT ON;
    SET XACT_ABORT ON;

    IF @GodineIskustva < 0 OR @GodineIskustva > 50
    BEGIN
        THROW 50001,
            N'Godine iskustva moraju biti između 0 i 50.',
            1;
    END;

    IF NULLIF(LTRIM(RTRIM(@ImePrezime)), N'') IS NULL
    BEGIN
        THROW 50002,
            N'Ime i prezime su obavezni.',
            1;
    END;
END;
```

```

        1;
END;

IF NULLIF(LTRIM(RTRIM(@Email)), N'') IS NULL
BEGIN
    THROW 50003,
        N'E-mail adresa je obavezna.',
        1;
END;

IF NULLIF(LTRIM(RTRIM(@PoslednjaSkola)), N'') IS NULL
BEGIN
    THROW 50004,
        N'Poslednja završena škola je obavezna.',
        1;
END;

IF NULLIF(LTRIM(RTRIM(@Zanimanje)), N'') IS NULL
BEGIN
    THROW 50005,
        N'Stečeno zanimanje je obavezno.',
        1;
END;

IF NULLIF(LTRIM(RTRIM(@RadnoMesto)), N'') IS NULL
BEGIN
    THROW 50006,
        N'Radno mesto je obavezno.',
        1;
END;

INSERT INTO dbo.PrijavaKandidata
(
    KorisnikID,
    ImePrezime,
    Email,
    KontaktTelefon,
    PoslednjaSkola,
    MestoSkole,
    Zanimanje,
    StepenObrazovanja,
    NazivKonkursa,
    RadnoMesto,
    GodineIskustva,
    MotivacionoPismo,
    DatumPodnosenja,
    RokKonkursa,
    StatusZahteva,

```

```

        IspunjavaOsnovneUslove
    )
VALUES
(
    @KorisnikID,
    LTRIM(RTRIM(@ImePrezime)),
    LTRIM(RTRIM(@Email)),
    LTRIM(RTRIM(@KontaktTelefon)),
    LTRIM(RTRIM(@PoslednjaSkola)),
    NULLIF(LTRIM(RTRIM(@MestoSkole)), N''),
    LTRIM(RTRIM(@Zanimanje)),
    LTRIM(RTRIM(@StepenObrazovanja)),
    LTRIM(RTRIM(@NazivKonkursa)),
    LTRIM(RTRIM(@RadnoMesto)),
    @GodineIskustva,
    NULLIF(LTRIM(RTRIM(@MotivacionoPismo)), N''),
    @DatumPodnosenja,
    @RokKonkursa,
    @StatusZahteva,
    @IspunjavaOsnovneUslove
);

SELECT
    CAST(SCOPE_IDENTITY() AS INT)
        AS ZahtevID;
END;

```

Listing 10. – Primer rada sa stored procedurama

5.6.1.3. Rad sa DBUtils i primena upita

DBUtils klase koriste se za rad sa SQL upitima i DataSet objektima.

```
using System.Data;
using DBUtils;

namespace KlasePodataka
{
    public class DBUtilsTipDokumentaKlasa : TabelaKlasa
    {
        public DBUtilsTipDokumentaKlasa(KonekcijaKlasa konekcija)
            : base(konekcija, "TipDokumenta")
        {
        }

        public DataSet DajSveTipoveDokumenata()
        {
            string upit = "SELECT * FROM TipDokumenta";
            return DajPodatke(upit);
        }

        public DataSet DajObavezneTipoveDokumenata()
        {
            string upit = "SELECT * FROM TipDokumenta WHERE Obavezan = 1";
            return DajPodatke(upit);
        }
    }
}
```

Listing 11. – Rad sa DBUtils

5.6.1.4. Rad sa Entity Framework klasama

Entity Framework korišćen je kroz DbContext klase i model korisnika.

```
using System.Data.Entity;

namespace KlasePodataka
{
    public class KonkursiZaPosaoContext :
        DbContext
    {
        public KonkursiZaPosaoContext()
            : base("name=KonkursiZaPosaoEF")
        {
            Database.SetInitializer
                <KonkursiZaPosaoContext>(
                    null
                );
        }

        public DbSet<KorisnikKlasa> Korisnici
        {
            get;
            set;
        }
    }
}
```

Listing 12. – Rad sa Entity Framework klasama

5.7. Master-detail odnos podataka

5.8. JavaScript validacije i regularni izrazi

JavaScript validacije realizovane su za sva polja

```
var imePrezime =  
    dajInput("ImePrezime");  
  
var email =  
    dajInput("Email");  
  
var kontaktTelefon =  
    dajInput("KontaktTelefon");  
  
var poslednjaSkola =  
    dajInput("PoslednjaSkola");  
  
var mestoSkole =  
    dajInput("MestoSkole");  
  
var zanimanje =  
    dajInput("Zanimanje");  
  
var stepenObrazovanja =  
    dajSelect("StepenObrazovanja");  
  
var nazivKonkursa =  
    dajInput("NazivKonkursa");  
  
var radnoMesto =  
    dajInput("RadnoMesto");  
  
var godineIskustva =  
    dajInput("GodineIskustva");  
  
var rokKonkursa =  
    dajInput("RokKonkursa");  
  
var motivacionoPismo =  
    dajTextarea("MotivacionoPismo");  
  
if (imePrezime !== null) {  
    var vrednostImena =  
        imePrezime.value.trim();
```



```

var regexImePrezime =
    /^[A-Za-zČĆŽŠĐčćžšđ]+(?:[ '-][A-Za-zČĆŽŠĐčćžšđ])+$/;

if (vrednostImena === "") {
    postaviGresku(
        imePrezime,
        "ImePrezime",
        "Unesite ime i prezime.",
        greske
    );

    prvoNeispravnoPolje =
        prvoNeispravnoPolje || imePrezime;
}
else if (
    vrednostImena.length < 5
    ||
    !regexImePrezime.test(vrednostImena)
) {
    postaviGresku(
        imePrezime,
        "ImePrezime",
        "Unesite puno ime i prezime, na primer: Marko Marković.",
        greske
    );

    prvoNeispravnoPolje =
        prvoNeispravnoPolje || imePrezime;
}
else if (vrednostImena.length > 100) {
    postaviGresku(
        imePrezime,
        "ImePrezime",
        "Ime i prezime mogu imati najviše 100 karaktera.",
        greske
    );

    prvoNeispravnoPolje =
        prvoNeispravnoPolje || imePrezime;
}
else {
    oznaciKaoIspravno(
        imePrezime
    );
}
}

```

Listing 13. – JavaScript validacije